Project Title

Dictionary-Based Fuzzy Search & Spell Correction Engine for Smart Shopping

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
J. Bhanu (2520030006)
Akshita A. (2520030043)
P. Harini (2520030177)

Team Number:

Section:

Under the guidance of

<<Guide Name>>



<<Designation, Department>>

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Dictionary-Based Fuzzy Search & Spell Correction Engine for Smart Shopping"** is a record of Bonafide work of **J. Bhanu – 2520030006, Akshita A. – 2520030043, P. Harini – 2520030177** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

J. Bhanu – 2520030006

Akshita A. – 2520030043

P. Harini – 2520030177

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Dictionary-Based Fuzzy Search & Spell Correction Engine for Smart Shopping"** is a bonafide work done and submitted by **J. Bhanu – 2520030006, Akshita A. – 2520030043, P. Harini – 2520030177** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, <<Guide Name>>, <<Designation>>, <Department>> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

J. Bhanu – 2520030006

Akshita A. – 2520030043

P. Harini – 2520030177

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	17
8.	Conclusion & Future Scope	24
9.	References	25

1.ABSTRACT

The Dictionary-Based Fuzzy Search & Spell Correction Engine for Smart Shopping is a console-based Java application that helps a shopper find products even when the product name is misspelled or only partly typed. Product details are loaded from a CSV catalog (products.csv) and the product names are indexed in a Trie. Fuzzy matching is done by walking the Trie while filling the Levenshtein edit-distance table row by row using dynamic programming, so work on shared prefixes is reused and branches that exceed the allowed distance are pruned. The system supports spell-corrected search ("Did you mean ...?"), prefix autocomplete, keyword search using the Rabin-Karp algorithm, category browsing, price-range search using binary search over price-sorted products, Top-N cheapest or most expensive products using a heap-based priority queue, trending searches from the session search history, and a shopping cart with a running total. HashMap, ArrayList and TreeSet are used for fast lookups, collections and sorted categories, and a menu option lists all the algorithms used. The project shows how classical data structures and algorithms solve a practical retrieval problem efficiently without needing a database.

Team Name:

Team Logo (if any):

Team Members:

1. J. Bhanu, 2520030006
2. Akshita A., 2520030043
3. P. Harini, 2520030177

2. INTRODUCTION

Online shopping has made the search box the main doorway to a product catalog. Shoppers type quickly on phones and keyboards, so names such as "iphn" for iPhone or "bskball" for basketball are common. A traditional exact-match search returns nothing for such queries, and the shopper may leave without buying. A good shopping search must therefore tolerate spelling mistakes, suggest completions while the user types, and still answer other everyday questions such as which items match a keyword, fall inside a budget, or are the cheapest in a category.

This project answers the what, why and how as follows. What: a Java console application, the Dictionary-Based Fuzzy Search & Spell Correction Engine for Smart Shopping, that loads a product catalog from a CSV file. Why: to show how core Data Structures and Algorithms from the course can be combined to solve a real problem efficiently. How: product names are stored in a Trie; Levenshtein distance computed with dynamic programming gives fuzzy search and spell correction; Rabin-Karp finds keywords; binary search handles price ranges; a heap-based priority queue returns Top-N products; and HashMap, ArrayList and TreeSet organise the data.

The rest of this report presents the literature survey, the problem statement and objectives, the hardware and software requirements, the implementation of the main modules, the results with output screenshots, and finally the conclusion, future scope and references.

3. LITERATURE SURVEY

Several existing tools and published approaches to spell correction and fuzzy search were studied before designing this project. The most relevant ones are summarised below, and their links are listed in the References section.

1. Norvig's Spelling Corrector. Peter Norvig's well-known essay shows how to build a spelling corrector in a few lines of code. It generates every word within one or two edits of the typed word, keeps the ones present in a dictionary, and picks the most frequent candidate using word counts from a large text. It is simple and effective, but generating all edits for every query becomes costly as the edit distance and word length grow.

2. Fuzzy Search Using a Trie and Levenshtein Distance. Steve Hanov describes storing the dictionary in a Trie and building the Levenshtein table one row per trie node while walking down the tree. Shared prefixes are computed only once and a branch is abandoned as soon as every value in the current row exceeds the maximum distance. This is the core technique used in our project for spell correction.

3. BK-Tree Based Approximate Matching. The Burkhard-Keller tree indexes words in a metric space such as edit distance. Each child edge stores the distance to its parent, and the triangle inequality lets a search skip whole subtrees that cannot contain a match. It is a good alternative index for fuzzy dictionary lookup, but it does not support prefix autocomplete, which the Trie gives us for free.

4. SymSpell. SymSpell is an open-source spelling-correction library that precomputes delete-only variants of every dictionary word, so that candidate lookup at query time needs no insertions, replacements or transpositions. It is very fast, but the precomputed index needs a lot of memory for larger edit distances.

5. Elasticsearch / Lucene Fuzzy Query. Elasticsearch offers a fuzzy query that returns documents containing terms within a given Levenshtein edit distance, using Levenshtein automata inside the Lucene index and a limit on the number of expansions. It shows how fuzzy matching is used in production search engines, which depend on a full search server rather than a small self-contained program.

6. Rabin-Karp Substring Search. The Rabin-Karp algorithm uses a rolling hash to compare a pattern against every window of a text in linear time on average, and it is widely used for keyword and multi-pattern matching. We use the same idea for keyword search over product names, and combine it with the Trie, binary search and heap modules to form a complete shopping search tool.

4. Problem Statement and Objectives

Problem Statement:

Exact-match product search fails when users mistype or partly type product names, and it gives no single way to search by keyword, category or price.

Problem Description:

Shoppers often misspell product names and get no results, so the system must find the intended product, suggest completions, and support keyword, category, price-range and Top-N queries efficiently using data structures and algorithms.

Objectives:

Project Purpose:

The project was started to show how data structures and algorithms can make a shopping search tolerant to spelling mistakes while staying fast. Based on this purpose, the objectives are:

1. To develop a smart shopping search engine in Java using Data Structures and Algorithms.

2. To give accurate product search results even when the user makes spelling mistakes.

3. To provide autocomplete suggestions for faster product searching.

4. To search products efficiently by name, keyword and category.

5. To provide price-range search so users can find products within a budget.

6. To find the cheapest and most expensive products efficiently using heap-based methods.

7. To keep track of frequently searched products using search history.

8. To provide basic shopping cart functionality for the selected products.

9. To demonstrate the practical use of DSA algorithms in a real-world shopping system.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (dual-core, 2.0 GHz or above)
- RAM: 4 GB minimum (8 GB recommended)
- Storage and peripherals: about 500 MB free disk space, standard keyboard and monitor

Software Requirements:

- Operating System: Windows 10 / 11 (Linux or macOS also work)
- Programming Language: Java, JDK 17 or above
- IDE / Editor: Visual Studio Code with a terminal (PowerShell) to compile and run
- Dataset and Tools: products.csv product catalog, Git and GitHub for version control

6. IMPLEMENTATION

Module 1: Trie (storage, exact search and autocomplete)

The Trie stores every product name letter by letter. insert() adds a name, containsExact() checks for an exact product name, and getWordsWithPrefix() returns up to a limit of names that start with the typed prefix, which powers autocomplete.

```
public class Trie {
    private final TrieNode root;
    private int wordCount = 0;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        if (word == null || word.isEmpty()) return;
        TrieNode current = root;
        for (char ch : word.toCharArray()) {
            current = current.children.computeIfAbsent(ch, c -> new TrieNode());
        }
        if (!current.isEndOfWord) {
            current.isEndOfWord = true;
            wordCount++;
        }
    }

    public boolean containsExact(String word) {
        TrieNode node = walkTo(word);
        return node != null && node.isEndOfWord;
    }

    private TrieNode walkTo(String word) {
        TrieNode current = root;
        for (char ch : word.toCharArray()) {
            current = current.children.get(ch);
            if (current == null) return null;
        }
        return current;
    }

    public List<String> getWordsWithPrefix(String prefix, int limit) {
        List<String> results = new ArrayList<>();
        TrieNode node = walkTo(prefix);
        if (node == null) return results;
        collectWords(node, new StringBuilder(prefix), results, limit);
        return results;
    }

    private void collectWords(TrieNode node, StringBuilder prefix, List<String> words,
int limit) {
        if (words.size() >= limit) return;
        if (node.isEndOfWord) {
            words.add(prefix.toString());
        }
        for (Map.Entry<Character, TrieNode> entry : node.children.entrySet()) {
            if (words.size() >= limit) return;
            prefix.append(entry.getKey());
```

```
            collectWords(entry.getValue(), prefix, words, limit);
            prefix.deleteCharAt(prefix.length() - 1);
        }
    }
}
```

Module 2: Fuzzy Search and Spell Correction (Levenshtein distance with dynamic programming)

The search() method walks the Trie depth-first and builds one row of the edit-distance table per node (buildNextRow). A word is accepted when its distance is within maxDistance, and a branch is pruned when every value in the row exceeds maxDistance. levenshteinDistance() is the standard full-table dynamic programming version.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Map;

public class FuzzySearchEngine {

    private final Trie dictionary;

    public FuzzySearchEngine(Trie dictionary) {
        this.dictionary = dictionary;
    }

    public List<Suggestion> search(String query, int maxDistance) {
        List<Suggestion> results = new ArrayList<>();

        int[] initialRow = new int[query.length() + 1];
        for (int i = 0; i <= query.length(); i++) {
            initialRow[i] = i;
        }

        StringBuilder currentWord = new StringBuilder();
        dfs(dictionary.getRoot(), query, initialRow, currentWord, maxDistance, results);

        Collections.sort(results);
        return results;
    }

    private void dfs(TrieNode node, String query, int[] previousRow,
                     StringBuilder currentWord, int maxDistance,
                     List<Suggestion> results) {

        int currentDistance = previousRow[query.length()];
        if (node.isEndOfWord && currentDistance <= maxDistance) {
            results.add(new Suggestion(currentWord.toString(), currentDistance));
        }

        if (min(previousRow) > maxDistance) {
            return;
        }

        for (Map.Entry<Character, TrieNode> entry : node.children.entrySet()) {
            char ch = entry.getKey();
```

```
            TrieNode child = entry.getValue();
            int[] currentRow = buildNextRow(previousRow, query, ch);

            currentWord.append(ch);
            dfs(child, query, currentRow, currentWord, maxDistance, results);
            currentWord.deleteCharAt(currentWord.length() - 1);
        }
    }

    private int[] buildNextRow(int[] previousRow, String query, char ch) {
        int[] currentRow = new int[query.length() + 1];
        currentRow[0] = previousRow[0] + 1;

        for (int i = 1; i <= query.length(); i++) {
            int insertCost = currentRow[i - 1] + 1;
            int deleteCost = previousRow[i] + 1;
            int substituteCost = previousRow[i - 1] + (query.charAt(i - 1) == ch ? 0 :
1);
            currentRow[i] = Math.min(Math.min(insertCost, deleteCost), substituteCost);
        }
        return currentRow;
    }

    private int min(int[] arr) {
        int m = arr[0];
        for (int v : arr) if (v < m) m = v;
        return m;
    }

    public static int levenshteinDistance(String a, String b) {
        int[][] dp = new int[a.length() + 1][b.length() + 1];
        for (int i = 0; i <= a.length(); i++) dp[i][0] = i;
        for (int j = 0; j <= b.length(); j++) dp[0][j] = j;

        for (int i = 1; i <= a.length(); i++) {
            for (int j = 1; j <= b.length(); j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                                    Math.min(dp[i][j - 1], dp[i - 1][j]));
                }
            }
        }
        return dp[a.length()][b.length()];
    }
}
```

Module 3: Price-Range Search (Binary Search)

With the products sorted by price, lowerBound() and upperBound() use binary search to find the first and last products inside the requested range, so the range is located in O(log n) time.

```
import java.util.ArrayList;
import java.util.List;

public class PriceSearch {
```

```
    public static List<Product> searchByRange(List<Product> sortedByPrice, double
minPrice, double maxPrice) {
        List<Product> results = new ArrayList<>();
        if (sortedByPrice.isEmpty() || minPrice > maxPrice) return results;

        int lower = lowerBound(sortedByPrice, minPrice);
        int upper = upperBound(sortedByPrice, maxPrice);

        for (int i = lower; i <= upper && i < sortedByPrice.size(); i++) {
            results.add(sortedByPrice.get(i));
        }
        return results;
    }

    private static int lowerBound(List<Product> list, double target) {
        int lo = 0, hi = list.size();
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (list.get(mid).price < target) {
                lo = mid + 1;
            } else {
                hi = mid;
            }
        }
        return lo;
    }

    private static int upperBound(List<Product> list, double target) {
        int lo = 0, hi = list.size();
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            if (list.get(mid).price <= target) {
                lo = mid + 1;
            } else {
                hi = mid;
            }
        }
        return lo - 1;
    }
}
```

Module 4: Top-N Cheapest / Most Expensive (Heap-based Priority Queue)

A bounded heap of size N is used: a max-heap keeps the N cheapest products and a min-heap keeps the N most expensive, so the whole catalog never has to be sorted.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

public class TopNFinder {

    public static List<Product> cheapest(List<Product> products, int n) {
        PriorityQueue<Product> maxHeap = new PriorityQueue<>(
                Comparator.comparingDouble((Product p) -> p.price).reversed());

        for (Product p : products) {
```

```
            maxHeap.offer(p);
            if (maxHeap.size() > n) {
                maxHeap.poll();
            }
        }
        List<Product> result = new ArrayList<>(maxHeap);
        result.sort(Comparator.comparingDouble(p -> p.price));
        return result;
    }

    public static List<Product> mostExpensive(List<Product> products, int n) {
        PriorityQueue<Product> minHeap = new PriorityQueue<>(
                Comparator.comparingDouble(p -> p.price));

        for (Product p : products) {
            minHeap.offer(p);
            if (minHeap.size() > n) {
                minHeap.poll();
            }
        }
        List<Product> result = new ArrayList<>(minHeap);
        result.sort(Collections.reverseOrder(Comparator.comparingDouble(p -> p.price)));
        return result;
    }
}
```

7. RESULTS & DISCUSSION

The application was compiled with javac and run from the terminal using java Main products.csv. The screenshots below show each feature working, with a short explanation.

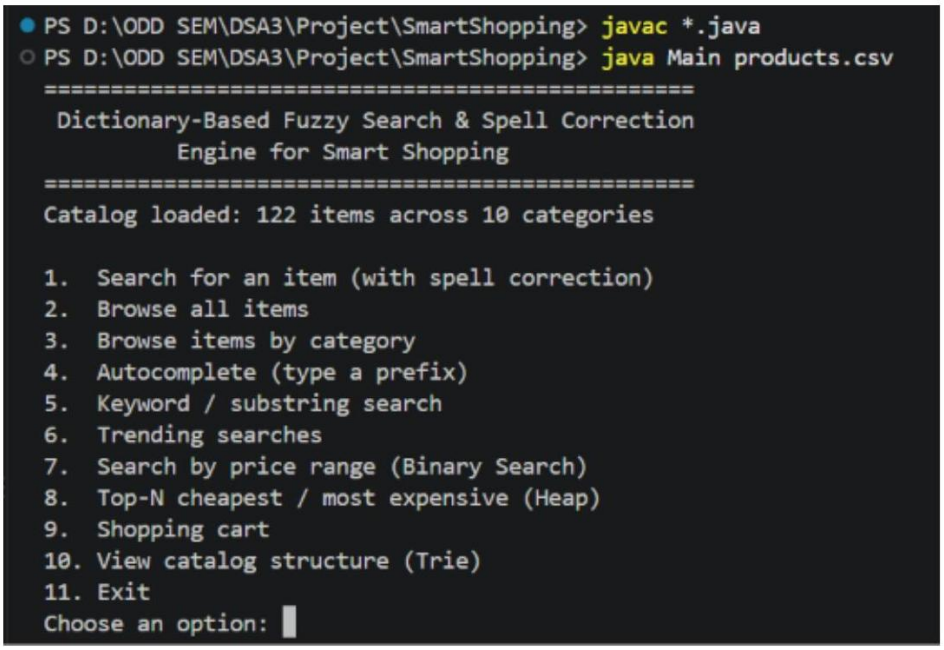


Fig. 1: Application start-up and main menu. The catalog is loaded into the Trie and the main menu of search, browse, price, Top-N, cart and Trie-view options is displayed.

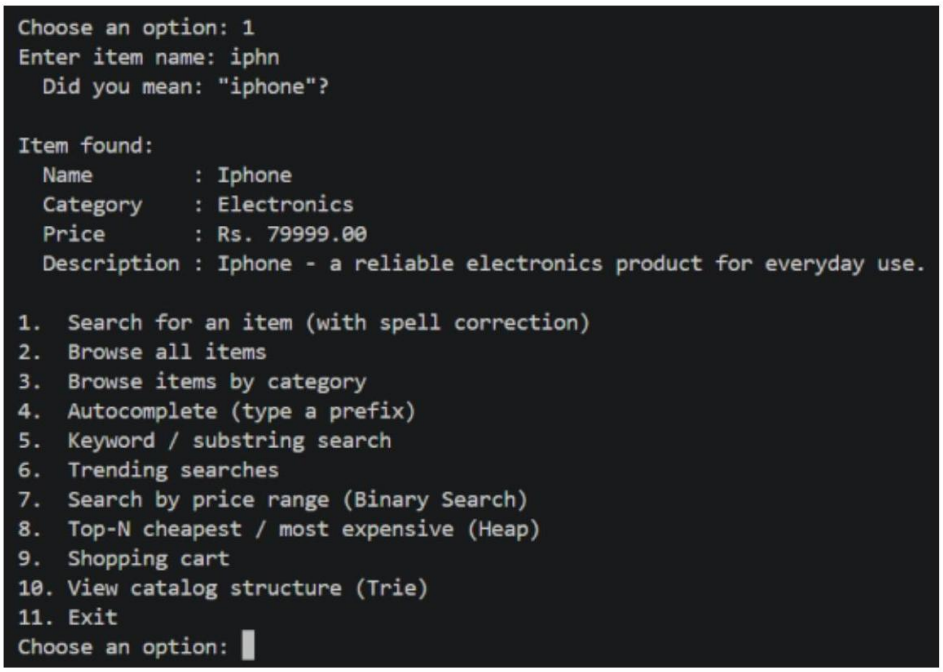


Fig. 2: Search with spell correction. The misspelled query "iphn" is corrected to "iphone" using the Trie and Levenshtein distance, and the product details are shown.



Fig. 3: Browse all items. All products in the catalog are listed with their name, category and price.

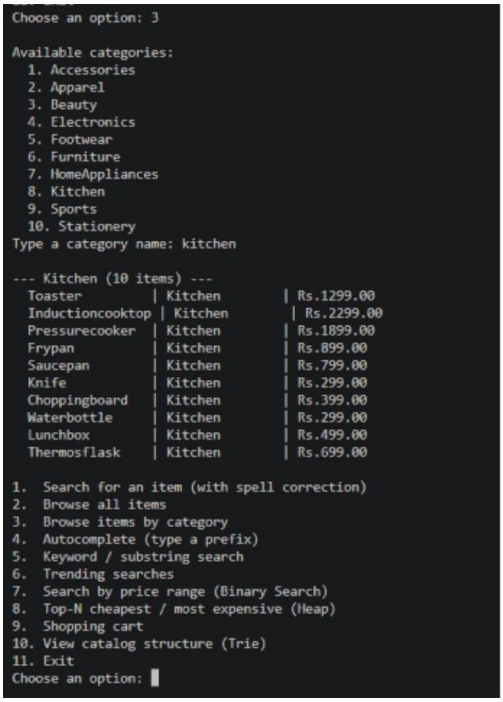


Fig. 4: Browse items by category. The categories, kept sorted in a TreeSet, are shown and the items of the chosen category (kitchen) are listed.

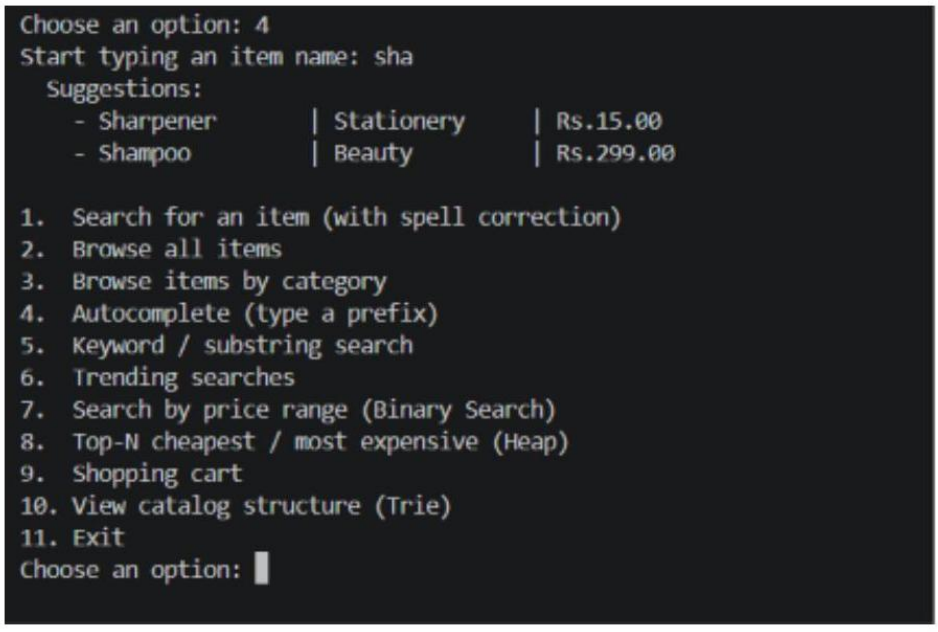


Fig. 5: Autocomplete. Typing the prefix "sha" returns the matching product suggestions (Sharpener, Shampoo) from the Trie.

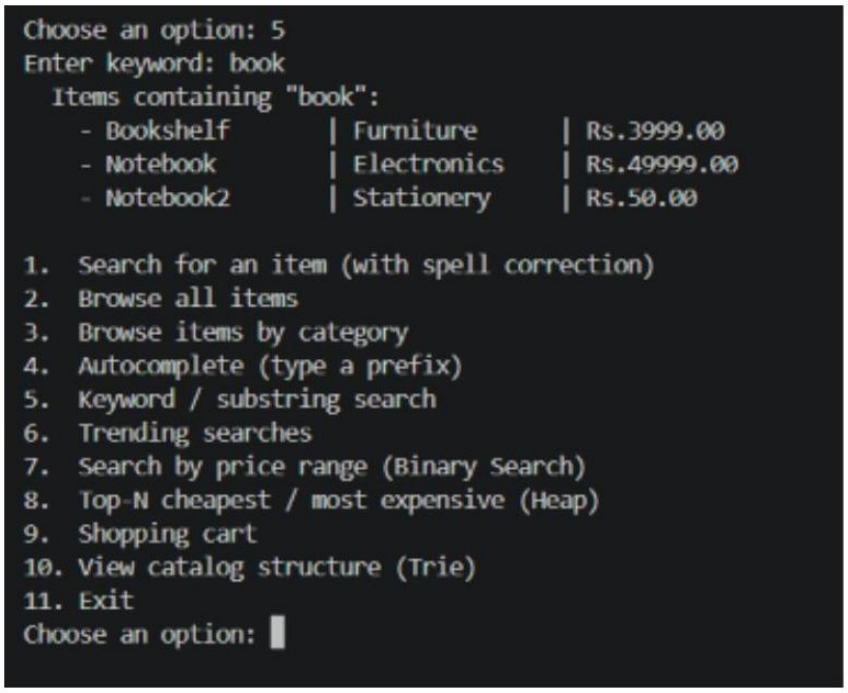


Fig. 6: Keyword search. The keyword "book" is found inside product names using the Rabin-Karp algorithm (Bookshelf, Notebook, Notebook2).

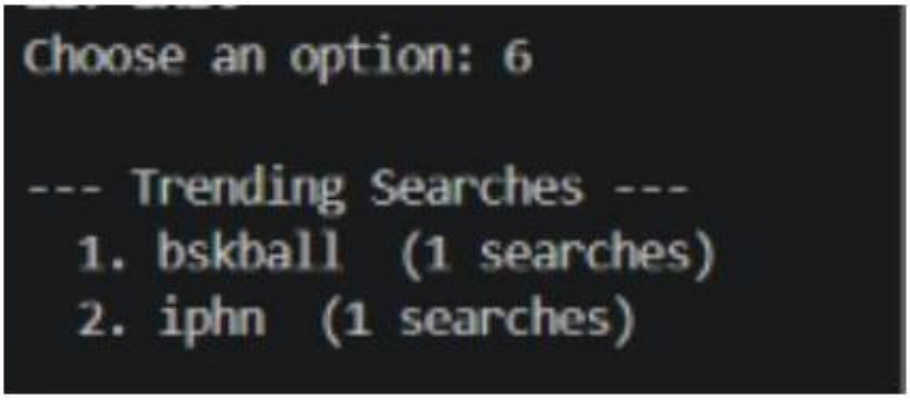


Fig. 7: Trending searches. Searches recorded during the session are ranked by how often they were made and shown as trending searches.



Fig. 8: Price-range search. Products priced between Rs.50 and Rs.2000 are retrieved using binary search over the price-sorted list.

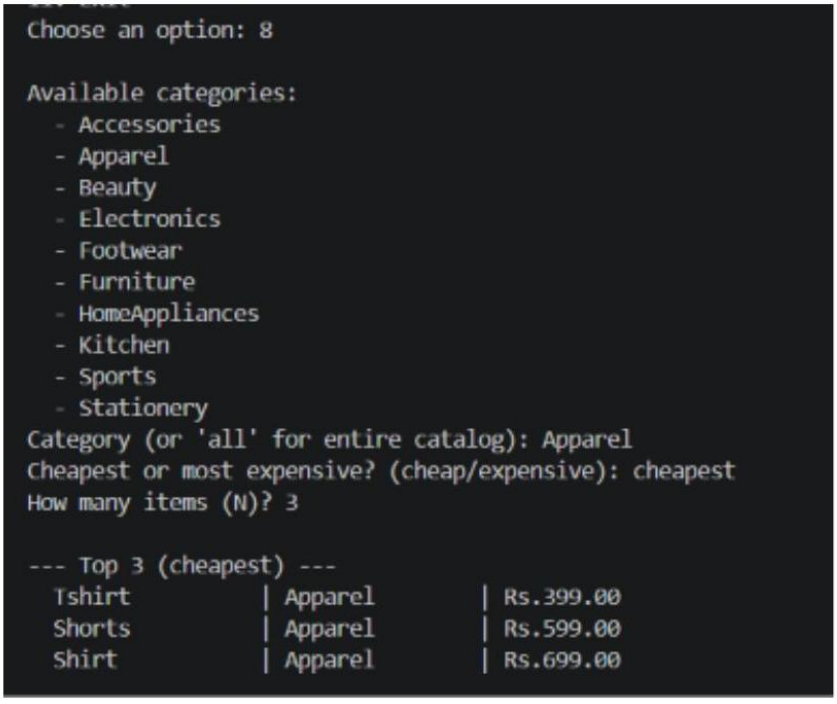


Fig. 9: Top-N cheapest items. The 3 cheapest items of the Apparel category are found using a heap-based priority queue.

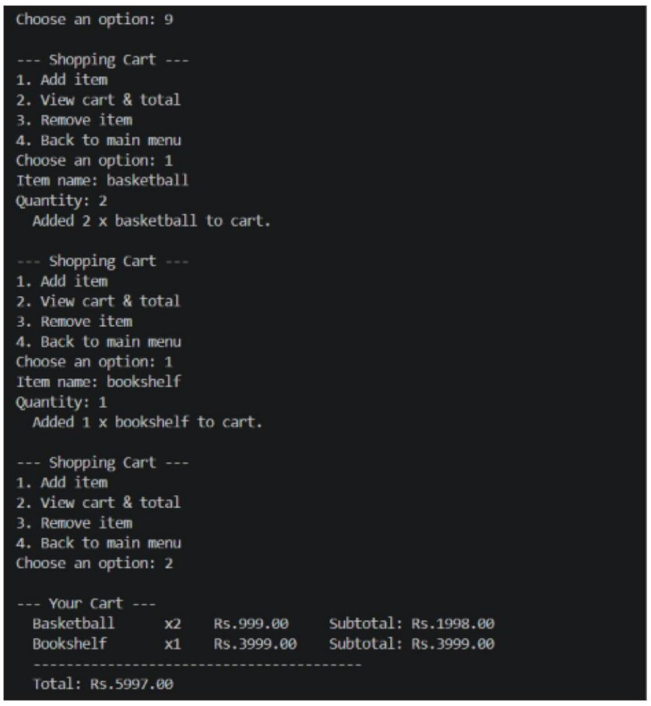


Fig. 10: Shopping cart - add and view. Items are added with quantities and the cart is displayed with the subtotal of each item and the total amount.

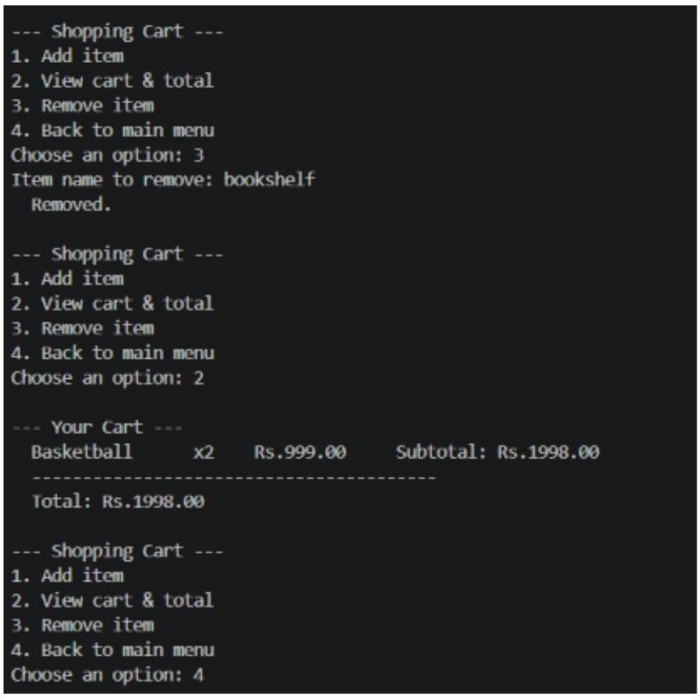


Fig. 11: Shopping cart - remove item. An item is removed from the cart and the cart is shown again with the updated total.



Fig. 12: Trie structure of the catalog. The Trie that stores the product names is printed level by level, with word-end markers.



Fig. 13: Exit. Choosing the exit option prints a thank-you message and returns to the terminal.

8. CONCLUSION & FUTURE SCOPE

The project met its objectives. A Java console application was built that corrects misspelled product names (for example "iphn" to "iphone") using a Trie and Levenshtein distance computed with dynamic programming, suggests products as the user types a prefix, finds keywords with the Rabin-Karp algorithm, filters products by price range with binary search, and returns the Top-N cheapest or most expensive items with a heap-based priority queue. Category browsing, trending searches and a shopping cart with a running total were also completed, all on top of a simple CSV product catalog. The work shows how different data structures and algorithms can be combined to build a fast and practical shopping search system.

Future Scope:

Near term: support transposed letters (Damerau-Levenshtein), correct whole multi-word queries, and rank suggestions by how popular each product is.

Medium term: add a graphical or web front end, store the catalog and search history in a database, and add filters such as brand and rating.

Long term: support phonetic and Indian-language search, give personalised recommendations from search history, and scale the index to very large catalogs.

9. REFERENCES

1. P. Norvig, "How to Write a Spelling Corrector" - https://norvig.com/spell-correct.html

2. S. Hanov, "Fast and Easy Levenshtein distance using a Trie" - https://stevehanov.ca/blog/index.php?id=114

3. Levenshtein distance (Wikipedia) - https://en.wikipedia.org/wiki/Levenshtein_distance

4. Trie (Wikipedia) - https://en.wikipedia.org/wiki/Trie

5. Rabin-Karp algorithm (Wikipedia) - https://en.wikipedia.org/wiki/Rabin%E2%80%93Karp_algorithm

6. BK-tree (Wikipedia) - https://en.wikipedia.org/wiki/BK-tree

7. SymSpell, spelling correction library (GitHub) - https://github.com/wolfgarbe/SymSpell

8. Elasticsearch fuzzy query documentation - https://www.elastic.co/guide/en/elasticsearch/reference/master/query-dsl-fuzzy-query.html

9. Binary search algorithm (Wikipedia) - https://en.wikipedia.org/wiki/Binary_search_algorithm

10. Priority queue / heap (Wikipedia) - https://en.wikipedia.org/wiki/Priority_queue